

Template for XCPC

Verifying We Are Human.

2025 ~ 2026 Chaoyi Sun, Liwen Zhang, Tian Zhou
2026 ~ 2027 Chaoyi Sun, Liwen Zhang, Fengting Deng

Contents

1	Graph	3
1.1	Diameter of a Tree	3
1.2	Centroid of a Tree	3
1.3	Minimum Spanning Tree	4
1.3.1	Prim	4
1.3.2	Kruskal	4
1.4	LCA	5
1.5	Topological Sorting	5
1.6	Shortest Path	6
1.6.1	Dijkstra	6
1.6.2	SPFA	6
1.7	DCC	7
1.8	Tarjan	8
1.9	Bipartite Graph Matchings	9
1.10	Flow	9
1.10.1	Edmonds-Karp	9
1.10.2	Dinic	10
2	Data Structure	14
2.1	Segment Tree	14
2.2	Fenwick Tree	14
2.3	Heavy-Light Decomposition	14
2.4	Splay Tree	14
2.5	Cartesian Tree	14
2.6	Sparse Table	14
2.7	Persistent Data Structure	14
2.8	Linear-Basis	14
3	Math	16
3.1	Subset Enumeration	16
3.2	Elementary Arithmetic Bucket	16
3.3	Fraction	16
3.4	Combination	17
3.5	The Sieve of Primes & Euler's Totient Function & Möbius Function	17
3.6	Exgcd	18
3.7	CRT	18
3.8	Lagrange Interpolation	18
3.9	Gaussian Elimination	19
3.10	FFT	19
3.11	NTT	20
3.12	FWT	22
4	Computational Geometry	23

5	String	30
5.1	Hash	30
5.2	KMP	30
5.3	Manacher	31
5.4	Trie	31
5.4.1	Trie	31
5.4.2	01-Trie	31
5.5	Aho-Corasick Automaton	32
5.6	SA	33
5.7	SAM	34
6	Other	36
6.1	Fast I/O	36
6.2	Random Number Generator	36
6.3	Execution	36
6.4	Comparsion	37
6.5	Python	37

1 Graph

1.1 Diameter of a Tree

```
1 template <typename T>
2 class Diameter
3 {
4     int n,p;
5     vector <T> dis;
6     vector <vector <pair <int,T>>> ve;
7     void dfs (int u,int fa)
8     {
9         for (const auto& [v,w] : ve[u])
10            {
11                if (v == fa) continue;
12                dis[v] = dis[u] + w;
13                if (dis[v] > dis[p]) p = v;
14                dfs (v,u);
15            }
16    }
17    public:
18    Diameter (int n) : n(n),ve(n + 1) {}
19    void add (int u,int v,T w = 1) {ve[u].push_back ({v,w});ve[v].push_back ({u,w});}
20    T calc ()
21    {
22        dis.assign (n + 1,0);
23        p = 1;dfs (1,0);
24        dis[p] = 0;dfs (p,0);
25        return dis[p];
26    }
27};
```

1.2 Centroid of a Tree

```
1 class Centroid
2 {
3     int n;
4     vector <int> sz,w,cen;
5     vector <vector <int>> ve;
6     void dfs (int u,int fa)
7     {
8         sz[u] = 1;w[u] = 0;
9         for (auto v : ve[u])
10            {
11                if (v == fa) continue;
12                dfs (v,u);
13                sz[u] += sz[v];
14                w[u] = max (w[u],sz[v]);
15            }
16        w[u] = max (w[u],n - sz[u]);
17        if (w[u] <= n / 2) cen.push_back (u);
18    }
19    public:
20    Centroid (int n) : n(n),ve (n + 1),sz (n + 1),w (n + 1) {}
21    void add (int u,int v) {ve[u].push_back (v);ve[v].push_back (u);}
22    auto calc ()
23    {
24        cen.clear ();
25        dfs (1, 0);
26        sort (cen.begin (),cen.end ());
```

```

27     return cen;
28 }
29 };

```

1.3 Minimum Spanning Tree

1.3.1 Prim

```

1 template <typename T>
2 class MST
3 {
4     int n;T ans;
5     vector <int> vis;vector <vector <int>> g;vector <T> dis;
6     public :
7     MST (int n) : n (n),vis (n + 1,0),g (n + 1,vector <int> (n + 1,INF)),dis (n +
8     1,INF) {dis[1] = ans = 0;vis[1] = 1;}
9     void add (int u,int v,T w) {g[u][v] = g[v][u] = w;}
10    T calc ()
11    {
12        for (int i = 2;i <= n;++i) dis[i] = g[1][i];
13        for (int i = 1;i < n;++i)
14        {
15            int k = 0;
16            for (int j = 1;j <= n;++j)
17                if (!vis[j] && dis[j] < dis[k]) k = j;
18            vis[k] = 1;
19            ans += dis[k];
20            for (int j = 1;j <= n;++j)
21                if (!vis[j] && g[k][j] < dis[j]) dis[j] = g[k][j];
22        }
23        return ans;
24    }
25 };

```

1.3.2 Kruskal

```

1 template <typename T>
2 class MST
3 {
4     int n,m,e_cnt,cnt;T ans;
5     struct node {int u,v;T w;};
6     vector <int> fa;vector <node> g;
7     public:
8     MST (int n,int m) : n (n),m(m),fa (n + 1,0),g (m + 1) {cnt = e_cnt = ans = 0;}
9     void add (int u,int v,int w) {g[++e_cnt].u = u,g[e_cnt].v = v,g[e_cnt].w = w;}
10    int getfa (int u) {return fa[u] == u ? u : fa[u] = getfa (fa[u]);}
11    T calc ()
12    {
13        sort (g.begin (),g.end (),[] (auto &x,auto &y) {return x.w < y.w;});
14        for (int i = 1;i <= n;++i) fa[i] = i;
15        for (int i = 1;cnt < n && i <= m;++i)
16        {
17            int dx = getfa (g[i].u),dy = getfa (g[i].v);
18            if (dx == dy) continue;
19            ans += g[i].w;fa[dx] = dy;++cnt;
20        }
21        return ans;
22    }
23 };

```

1.4 LCA

```
1 class LCA
2 {
3     static constexpr int lg = 20;
4     int n;
5     vector<int> dep;
6     vector<vector<int>> f, ve;
7     public:
8     LCA (int n) : n (n), ve (n + 1), dep (n + 1), f (n + 1, vector<int> (lg + 1, 0)) {}
9     void add (int u, int v) {ve[u].push_back (v); ve[v].push_back (u);}
10    void pre (int u, int fa)
11    {
12        f[u][0] = fa; dep[u] = dep[fa] + 1;
13        for (int i = 0; i < lg; ++i) f[u][i + 1] = f[f[u][i]][i];
14        for (auto v : ve[u])
15            if (v != fa) pre (v, u);
16    }
17    int query (int u, int v)
18    {
19        if (dep[u] < dep[v]) swap (u, v);
20        for (int i = lg; ~i; --i)
21        {
22            if (dep[f[u][i]] >= dep[v]) u = f[u][i];
23            if (u == v) return u;
24        }
25        for (int i = lg; ~i; --i)
26            if (f[u][i] != f[v][i]) u = f[u][i], v = f[v][i];
27        return f[u][0];
28    }
29 };
```

1.5 Topological Sorting

```
1 class Topo
2 {
3     int n;
4     vector<int> deg;
5     vector<vector<int>> ve;
6
7     public:
8     Topo (int n) : n (n), ve (n + 1), deg (n + 1, 0) {}
9     void add (int u, int v)
10    {
11        ve[u].push_back (v);
12        ++deg[v];
13    }
14    vector<int> calc ()
15    {
16        queue<int> q;
17        vector<int> lst;
18        for (int i = 1; i <= n; ++i)
19            if (!deg[i]) q.push (i);
20        while (!q.empty ())
21        {
22            int u = q.front (); q.pop ();
23            lst.push_back (u);
24            for (auto v : ve[u])
25            {
26                --deg[v];
27                if (!deg[v]) q.push (v);
28            }
29        }
30    }
```

```

29     }
30     return lst;
31 }
32 };

```

1.6 Shortest Path

1.6.1 Dijkstra

```

1 class dijkstra
2 {
3     int n,m,cnt;
4     vector <int> head,to,nxt,vis;
5     vector <ll> dis,val;
6     public:
7     dijkstra (int n,int m) :
8         n (n),m (m),vis (n + 1,0),head (n + 1,0),dis (n + 1,1e18),
9         to (2 * m + 1,0),nxt (2 * m + 1,0),val (2 * m + 1,0) {cnt = 0;}
10    void add (int u,int v,int w)
11    {
12        to[++cnt] = v;val[cnt] = w;nxt[cnt] = head[u];head[u] = cnt;
13        to[++cnt] = u;val[cnt] = w;nxt[cnt] = head[v];head[v] = cnt;
14    }
15    auto calc (int s)
16    {
17        priority_queue <pair <ll,int>> q;
18        for (int i = 1;i <= n;++i) vis[i] = 0,dis[i] = INF;
19        q.push ({0,s});
20        dis[s] = 0;
21        while (!q.empty ())
22        {
23            int u = q.top ().second;q.pop ();
24            if (vis[u]) continue;
25            vis[u] = 1;
26            for (int i = head[u];i;i = nxt[i])
27            {
28                int v = to[i];
29                if (dis[v] > dis[u] + val[i])
30                {
31                    dis[v] = dis[u] + val[i];
32                    q.push ({-dis[v],v});
33                }
34            }
35        }
36        return dis;
37    }
38 };

```

1.6.2 SPFA

```

1 class SPFA
2 {
3     int n,m,cnt;
4     vector <int> head,to,nxt,val,vis,times;
5     vector <ll> dis;
6     public:
7     SPFA (int n,int m) :
8         n (n),m (m),times (n + 1,0),vis (n + 1,0),head (n + 1,0),dis (n + 1,INF),
9         to (2 * m + 1,0),nxt (2 * m + 1,0),val (2 * m + 1,0) {cnt = 0;}
10    void add (int u,int v,int w)

```

```

11 {
12     to[++cnt] = v;val[cnt] = w;nxt[cnt] = head[u];head[u] = cnt;
13     to[++cnt] = u;val[cnt] = w;nxt[cnt] = head[v];head[v] = cnt;
14 }
15 auto calc (int s)
16 {
17     queue <int> q;
18     for (int i = 1;i <= n;++i) vis[i] = 0,dis[i] = INF;
19     dis[s] = 0,vis[s] = 1;q.push (s);
20     while (!q.empty())
21     {
22         int u = q.front ();
23         q.pop (),vis[u] = 0;
24         for (int i = head[u];i;i = nxt[i])
25         {
26             int v = to[i];
27             if (dis[v] > dis[u] + val[i])
28             {
29                 dis[v] = dis[u] + val[i];
30                 times[v] = times[u] + 1;
31                 if (times[v] >= n) return {-1}; //Negative Cycle
32                 if (!vis[v]) q.push (v),vis[v] = 1;
33             }
34         }
35     }
36     return dis;
37 }
38 };

```

1.7 DCC

```

1 class DCC
2 {
3     public:
4     struct Edge {int to,nxt;};
5     vector <Edge> G;
6     vector <int> head,dfn,low,col;
7     vector <bool> cut;
8     stack <int> stk;
9     int cnt,cct,n;
10    void init (int nn = 0)
11    {
12        G.assign (2,{0,0});
13        n = nn;head.assign (n + 1,0);
14    }
15    void dfs (int u,int f = -1)
16    {
17        dfn[u] = low[u] = ++cnt;
18        int ch = 0;
19        for (int i = head[u];i;i = G[i].nxt)
20            if (i != f)
21            {
22                int v = G[i].to;
23                if (dfn[v] < dfn[u]) stk.push (i >> 1);
24                if (!dfn[v])
25                {
26                    dfs (v,i ^ 1);++ch;
27                    low[u] = min (low[u],low[v]);
28                    if (low[v] >= dfn[u])
29                    {
30                        int I = 0;
31                        ++cct;cut[u] = true;

```

```

32         do
33         {
34             assert (!stk.empty ());
35             I = stk.top ();stk.pop ();
36             col[I] = cct;
37         }
38         while (I != (i >> 1));
39     }
40 }
41     else if (dfn[v] < low[u]) low[u] = dfn[v];
42 }
43     if (f == -1 && ch == 1) cut[u] = false;
44 }
45 void tarjan ()
46 {
47     cnt = cct = 0;
48     col.assign (G.size () >> 1,0);
49     dfn.assign (n + 1,0);
50     low.assign (n + 1,0);
51     cut.assign (n + 1,false);
52     while (!stk.empty ()) stk.pop ();
53     for (int i = 1;i <= n;++i) if (!dfn[i]) dfs (i);
54 }
55 void insert (int u,int v)
56 {
57     G.push_back ({v,head[u]});head[u] = G.size () - 1;
58     G.push_back( {u,head[v]});head[v] = G.size () - 1;
59 }
60 void insert (vector <vector <int>> &G1)
61 {
62     for (unsigned u = 1;u < G1.size ();++u)
63         for (int v : G1[u]) insert (u,v);
64 }
65 bool operator [] (const int &x) const {return cut[x];}
66 };

```

1.8 Tarjan

```

1 class Tarjan
2 {
3     int n,m,cnt,times,scc_cnt;
4     vector <int> head,to,nxt,low,scc,dfn;
5     stack <int> s;
6     void tarjan (int u)
7     {
8         low[u] = dfn[u] = ++times;
9         s.push (u);
10        for (int i = head[u];i;i = nxt[i])
11        {
12            int v = to[i];
13            if (!dfn[v])
14            {
15                tarjan (v);
16                low[u] = min (low[u],low[v]);
17            }
18            else if (!scc[v]) low[u] = min (low[u],dfn[v]);
19        }
20        if (low[u] == dfn[u])
21        {
22            ++scc_cnt;
23            while (1)
24            {

```



```

25         int x = s.top ();s.pop ();
26         scc[x] = scc_cnt;
27         if (x == u) break;
28     }
29 }
30 }
31 public:
32 Tarjan (int n,int m) :
33     n (n),m (m),head (n + 2,0),low (n + 1,0),dfn (n + 1,0),scc (n + 1,0),
34     to (2 * m + 2,0),nxt (2 * m + 2,0) {cnt = 1;times = scc_cnt = 0;}
35 void add (int u,int v) {to[++cnt] = v;nxt[cnt] = head[u];head[u] = cnt;}
36 auto calc ()
37 {
38     for (int i = 1;i <= n;++i)
39         if (!dfn[i]) tarjan (i);
40     return scc;
41 }
42 };

```

1.9 Bipartite Graph Matchings

```

1 class Matching
2 {
3     int l,r;//the number of left/right side points
4     vector <vector <int>> ve;
5     vector <int> vis,op;
6     bool dfs (int u)
7     {
8         for (auto v : ve[u])
9         {
10             if (vis[v]) continue;
11             vis[v] = 1;
12             if (!op[v] || dfs (op[v])) {op[v] = u;return true;}
13         }
14         return false;
15     }
16 public:
17 Matching (int l,int r) : l (l),r (r),vis (r + 1,0),op (r + 1,0),ve (l + 1) {}
18 void add (int u,int v) {ve[u].push_back (v);}
19 int calc ()
20 {
21     int ans = 0;
22     for (int i = 1;i <= l;++i)
23     {
24         vis.assign (r + 1,0);
25         if (dfs (i)) ++ans;
26     }
27     return ans;
28 }
29 };

```

1.10 Flow

1.10.1 Edmonds-Karp

```

1 template <typename T>
2 class EK
3 {
4     int n,m,s,t,cnt;
5     vector <int> head,to,nxt,vis,pre,edge;

```

```

6   vector <T> val;
7   bool bfs ()
8   {
9       queue <T> q;
10      for (int i = 1; i <= n; ++i) vis[i] = 0, pre[i] = edge[i] = -1;
11      vis[s] = 1; q.push (s);
12      while (!q.empty ())
13      {
14          int u = q.front (); q.pop ();
15          for (int i = head[u]; i; i = nxt[i])
16          {
17              int v = to[i];
18              if (!vis[v] && val[i])
19              {
20                  pre[v] = u; edge[v] = i; vis[v] = 1;
21                  q.push (v);
22                  if (v == t) return 1;
23              }
24          }
25      }
26      return 0;
27  }
28  public :
29  EK (int n, int m, int s, int t) :
30      n (n), m (m), s (s), t (t),
31      vis (n + 1, 0), head (n + 1, 0), pre (n + 1, -1), edge (n + 1, -1),
32      to (m + 1, 0), nxt (m + 1, 0), val (m + 1, 0) {cnt = 1;}
33  void add (int u, int v, T w)
34  {
35      to[++cnt] = v; val[cnt] = w; nxt[cnt] = head[u]; head[u] = cnt;
36      to[++cnt] = u; val[cnt] = 0; nxt[cnt] = head[v]; head[v] = cnt;
37  }
38  T calc ()
39  {
40      T ans = 0;
41      while (bfs ())
42      {
43          T mn = INF;
44          for (int i = t; i != s; i = pre[i]) mn = min (mn, val[edge[i]]);
45          for (int i = t; i != s; i = pre[i]) val[edge[i]] -= mn, val[edge[i] ^ 1]
+= mn;
46          ans += mn;
47      }
48      return ans;
49  }
50 };

```

1.10.2 Dinic

```

1  template <typename T>
2  class Dinic
3  {
4      int n, m, s, t, cnt;
5      vector <int> head, to, nxt, cur, dep;
6      vector <T> val;
7      int bfs ()
8      {
9          for (int i = 0; i <= n; ++i) dep[i] = 0, cur[i] = head[i];
10         queue <int> q;
11         q.push (s), dep[s] = 1;
12         while (!q.empty ())
13         {

```

```

14         int u = q.front ();q.pop ();
15         for (int i = head[u];i;i = nxt[i])
16         {
17             int v = to[i];
18             if (val[i] && !dep[v]) q.push (v),dep[v] = dep[u] + 1;
19         }
20     }
21     return dep[t];
22 }
23 T dfs (int u,int t,T flow)
24 {
25     if (u == t) return flow;
26     T ans = 0;
27     for (int &i = cur[u];i && ans < flow;i = nxt[i])
28     {
29         int v = to[i];
30         if (val[i] && dep[v] == dep[u] + 1)
31         {
32             int x = dfs (v,t,min (val[i], flow - ans));
33             if (x) val[i] -= x,val[i ^ 1] += x,ans += x;
34         }
35     }
36     if (ans < flow) dep[u] = -1;
37     return ans;
38 }
39 public :
40 Dinic (int n,int m,int s,int t) :
41     n (n),m (m),s (s),t (t),
42     head (n + 1,0),cur (n + 1,0),dep (n + 1,0),
43     to (m + 1,0),nxt (m + 1,0),val (m + 1,0) {cnt = 1;}
44 void add (int u,int v,T w)
45 {
46     to[++cnt] = v;val[cnt] = w;nxt[cnt] = head[u];head[u] = cnt;
47     to[++cnt] = u;val[cnt] = 0;nxt[cnt] = head[v];head[v] = cnt;
48 }
49 T calc ()
50 {
51     T ans = 0;
52     while (bfs ())
53     {
54         T x;
55         while ((x = dfs (s,t,INF))) ans += x;
56     }
57     return ans;
58 }
59 };

```

```

1 class Dinic
2 {
3     typedef long long ll;
4     static const int N=1e6+10,M=4e6+10;
5     static const ll INF=4e18;
6     int cgt,head[N],des[M],nxt[M],dep[N],que[N],H,T,h2[N];
7     ll cap[M];
8     inline ll dfs(const int &u,ll in=INF)
9     {
10         if(u==t)return in;ll out=0;
11         for(int &i=h2[u];i;i=nxt[i])
12         {
13             int v=des[i];
14             if(cap[i]&&dep[v]==dep[u]+1)
15             {
16                 ll tmp=dfs(v,min(in,cap[i]));

```

```

17         out+=tmp;cap[i]-=tmp;cap[i^1]+=tmp;
18         if(!(in-=tmp))break;
19     }
20 }
21 return out?out:dep[u]=0;
22 }
23 inline bool bfs()
24 {
25     memset(dep+1,0,n*sizeof(ll));
26     dep[que[H=T=0]]=s=1;
27     while(H<=T)
28     {
29         int u=que[H++];
30         for(int i=head[u];i;i=nxt[i])if(cap[i])
31         {
32             int v=des[i];
33             if(!dep[v])dep[que[++T]=v]=dep[u]+1;
34         }
35     }
36     return dep[t];
37 }
38 public:
39     int n,s,t;
40     inline void Init(const int &sz=0)
41     {
42         memset(head+1,0,n*sizeof(ll));
43         cgt=1;n=sz;
44     }
45     inline ll Max_Flow(const int &ss,const int &tt)
46     {
47         s=ss;t=tt;ll ans=0;
48         while(bfs())
49         {
50             memcpy(h2+1,head+1,n*sizeof(ll));
51             ans+=dfs(s,INF);
52         }
53         return ans;
54     }
55     inline bool ins(const int &x,const int &y,const ll &w)
56     {
57         if(x>n||y>n||x<1||y<1)return false;
58         nxt[++cgt]=head[x];des[head[x]=cgt]=y;cap[cgt]=w;
59         nxt[++cgt]=head[y];des[head[y]=cgt]=x;cap[cgt]=0;
60         return true;
61     }
62     inline int newNode(){return ++n;}
63 }a;
64
65 namespace MCMF
66 {
67     typedef long long Val_Type;typedef long long Cap_Type;
68     const Val_Type INF_v=4e18;const Cap_Type INF_c=4e18;
69     const int N=1e6+10,M=2e6+10;int head[N],des[M],nxt[M],cgt=1;Cap_Type cap[M];
70     Val_Type val[M];
71     template<typename T> inline T Abs(T x){return x<0?-x:x;}
72     inline void ins(const int &x,const int &y,const int &c,const Val_Type &w)
73     {
74         nxt[++cgt]=head[x];des[head[x]=cgt]=y;cap[cgt]=c;val[cgt]=w;
75         nxt[++cgt]=head[y];des[head[y]=cgt]=x;cap[cgt]=0;val[cgt]=-w;
76     }
77     int que[20000010],H,T;Val_Type dis[N],mnc;int s,t,S;bool inq[N];
78     inline bool spfa()

```

```

79 {
80     for(int i=0;i<=S+2;++i)dis[i]=INF_v;dis[que[H=T=0]]=s=0;
81     while(H<=T)
82     {
83         int u=que[H++];inq[u]=false;for(int i(head[u]);i;i=nxt[i])
84             {int v=des[i];if(cap[i]&&dis[v]>dis[u]+val[i]){dis[v]=dis[u]+val[i]
];if(!inq[v])inq[que[++T]=v]=true;}}
85     }
86     return dis[t]<INF_v;
87 }
88 inline Cap_Type dfs(const int &u=s, Cap_Type in=INF_c)
89 {
90     if(u==t)return in;Cap_Type out=0;inq[u]=true;
91     for(int i(head[u]);i;i=nxt[i])
92     {
93         int v=des[i];if(cap[i]&&!inq[v]&&dis[v]==(dis[u]+val[i]))
94         {
95             Cap_Type tmp=dfs(v, cap[i]<in?cap[i]:in);mnc+=val[i]*tmp;
96             cap[i]-=tmp;cap[i^1]+=tmp;out+=tmp;if(!(in-=tmp))break;
97         }
98     }
99     if(!out)dis[u]=4e18;inq[u]=false;return out;
100 }
101 inline Cap_Type dinic(){Cap_Type res=0;while(spfa())res+=dfs();return res;}
102 inline void Clear(){memset(head,0,(S+2)*sizeof(head[0]));cgt=1;S=0;mnc=0;}
103 }
104 using MCMF::ins;using MCMF::s;using MCMF::t;using MCMF::S;using MCMF::dinic;using
MCMF::mnc;

```

2 Data Structure

2.1 Segment Tree

2.2 Fenwick Tree

2.3 Heavy-Light Decomposition

2.4 Splay Tree

2.5 Cartesian Tree

```
1 //heap-order on values and BST-order on keys
2 stack <int> s;
3 vector <int> L (n + 1,0),R (n + 1,0);
4 for (int i = 1;i <= n;++i)
5 {
6     int la = -1;
7     while (!s.empty () && a[s.top ()] > a[i]) la = s.top (),s.pop ();
8     if (!s.empty ()) R[s.top ()] = i;
9     if (la != -1) L[i] = la;
10    s.push (i);
11 }
```

2.6 Sparse Table

```
1 vector <int> lg (n + 1,-1);
2 vector <vector <int>> f (20,vector <int> (n + 1,INF));
3 for (int i = 1;i <= n;++i) lg[i] = lg[i >> 1] + 1;
4 for (int i = 1;i <= n;++i) f[0][i] = a[i];
5 for (int j = 1;j < 20;++j)
6     for (int i = 1;i + (1 << j) - 1 <= n;++i) f[j][i] = min (f[j - 1][i],f[j - 1][i
7     + (1 << (j - 1))]);
8 auto query = [&] (int l,int r) -> int {return min (f[lg[r - l + 1]][l],f[lg[r - l +
9     1]][r - (1 << lg[r - l + 1]) + 1]);};
```

2.7 Persistent Data Structure

2.8 Linear-Basis

```
1 template <typename T>
2 class Basis
3 {
4     int n,lg;
5     vector <T> p,ex,nw;vector <vector <int>> id;
6     public :
7     Basis (int n,int lg) : n (n),lg (lg),p (lg + 1,0),nw (lg + 1,0),id (lg + 1) {}
8     void modify (T x,int idx)
9     {
10        vector <int> cur (n + 1,0);cur[idx] = 1;
11        for (int i = lg;~i;--i)
12        {
13            if (!((1ll << i) & x)) continue;
14            if (!p[i]) {p[i] = x;id[i] = cur;break;}
15            else
16            {
17                x ^= p[i];
18                for (int j = 1;j <= n;++j) cur[j] ^= id[i][j];
19            }
20        }
```

```

21 }
22 T get_max ()
23 {
24     T ans = 0;
25     for (int i = lg;~i;--i)
26         if ((ans ^ p[i]) > ans) ans ^= p[i];
27     return ans;
28 }
29 T get_min ()
30 {
31     T ans = 0;int cnt = 0;
32     for (int i = 0;i <= lg;++i) cnt += p[i] > 0;
33     if (cnt < n) return 0;
34     for (int i = 0;i <= lg;++i)
35         if (p[i] > 0) return p[i];
36 }
37 void change ()
38 {
39     for (int i = 0;i <= lg;++i) nw[i] = p[i];
40     for (int i = 0;i <= lg;++i)
41         for (int j = i - 1;~j;--j)
42             if (nw[i] >> j & 1) nw[i] ^= nw[j];
43     for (int i = 0;i <= lg;++i)
44         if (nw[i]) ex.push_back (i);
45 }
46 T get_min_k (T k)
47 {
48     int sz = (int)ex.size ();
49     if (sz < n) --k; // element 0
50     if (k > (1ll << sz) - 1) return -1;
51     T ans = 0;
52     for (int j = 0;j < sz;++j)
53         if (k >> j & 1) ans ^= nw[ex[j]];
54     return ans;
55 }
56 bool find (T x)
57 {
58     for (int i = lg;~i;--i)
59     {
60         if (!(x >> i & 1)) continue;
61         if (!p[i]) return false;
62         x ^= p[i];
63     }
64     return true;
65 }
66 vector <int> getid (T x) //if find (x) == 1
67 {
68     vector <int> res (n + 1,0);
69     for (int i = lg;~i;--i)
70     {
71         if (!(x >> i & 1)) continue;
72         x ^= p[i];
73         for (int j = 1; j <= n; ++j) res[j] ^= id[i][j];
74     }
75     return res;
76 }
77 };

```

3 Math

3.1 Subset Enumeration

```
1 for (int S = 1; S < (1 << n); ++S)
2     for (int S0 = S; S0; S0 = (S0 - 1) & S)
```

3.2 Elementary Arithmetic Bucket

```
1 template <typename T, int MOD = 1000000007>
2 class Z
3 {
4     T x;
5     Z <T,MOD> qpow (Z x, T y)
6     {
7         Z <T,MOD> res (1);
8         while (y)
9         {
10             if (y & 1) res *= x;
11             x *= x;
12             y >>= 1;
13         }
14         return res;
15     }
16     public :
17     Z () : x(0) {}
18     Z (T x) : x (x) {}
19     T get () {return x;}
20     Z & operator = (T o) {x = o % MOD; return *this;}
21     Z & operator += (Z o) {x = (x + o.x + MOD) % MOD; return *this;}
22     Z & operator -= (Z o) {x = (x - o.x + MOD) % MOD; return *this;}
23     Z & operator *= (Z o) {x = 1ll * o.x * x % MOD; return *this;}
24     Z & operator ^= (Z o) {x = qpow (x, o.x).x; return *this;}
25     Z & operator /= (Z o) {*this *= o ^ (MOD - 2); return *this;}
26     Z & operator %=(Z o) {x = x % o.x; return *this;}
27     friend Z operator + (Z x, Z y) {return x += y;}
28     friend Z operator - (Z x, Z y) {return x -= y;}
29     friend Z operator * (Z x, Z y) {return x *= y;}
30     friend Z operator / (Z x, Z y) {return x /= y;}
31     friend Z operator ^ (Z x, Z y) {return x ^= y;}
32     friend Z operator % (Z x, Z y) {return x %=(y);}
33     friend bool operator == (Z x, Z y) {return x.x == y.x;}
34     friend bool operator != (Z x, Z y) {return x.x != y.x;}
35     friend istream& operator >> (istream &is, Z& o) {T val; is >> val; o = Z (val);
36     return is;}
37     friend ostream& operator << (ostream &os, const Z &z) {return os << z.x;}
38 };
39
```

3.3 Fraction

```
1 template <typename T>
2 class Frac
3 {
4     T p,q; // p / q
5     void reduce ()
6     {
7         assert (q != 0);
8         T g = gcd (p,q);
9         p /= g, q /= g;
```



```

10     if (q < 0) p = -p,q = -q;
11 }
12 public :
13 Frac (T p = 0, T q = 1) : p (p),q (q) {assert (q != 0);reduce ();}
14 T get_p () {return p;}
15 T get_q () {return q;}
16 double get_val () {return 1.0 * p / q;}
17 Frac operator + (const Frac &o) {return Frac (p * o.q + o.p * q,q * o.q);}
18 Frac operator - (const Frac &o) {return Frac (p * o.q - o.p * q,q * o.q);}
19 Frac operator * (const Frac &o) {return Frac (p * o.p,q * o.q);}
20 Frac operator / (const Frac &o) {return Frac (p * o.q,q * o.p);}
21 Frac& operator += (const Frac &o) {*this = *this + o;return *this;}
22 Frac& operator -= (const Frac &o) {*this = *this - o;return *this;}
23 Frac& operator *= (const Frac &o) {*this = *this * o;return *this;}
24 Frac& operator /= (const Frac &o) {*this = *this / o;return *this;}
25 Frac operator -() {return Frac (-p,q);}
26 bool operator >= (const Frac &o) {return p * o.q >= o.p * q;}
27 bool operator > (const Frac &o) {return p * o.q > o.p * q;}
28 bool operator == (const Frac &o) {return p * o.q == o.p * q;}
29 bool operator < (const Frac &o) {return !(*this > o);}
30 bool operator <= (const Frac &o) {return !(*this >= o);}
31 bool operator != (const Frac &o) {return !(*this == o);}
32 };

```

3.4 Combination

```

1 template <typename T,int MOD = 1000000007>
2 class COM
3 {
4     int n;
5     vector <Z <T,MOD>> fac,inv;
6     public :
7     COM (int n) : n (n),fac (2 * n + 2),inv (2 * n + 2)
8     {
9         fac[0] = inv[0] = inv[1] = 1;
10        for (int i = 1;i <= 2 * n;++i) fac[i] = fac[i - 1] * i;
11        inv[2 * n] = fac[2 * n] ^ (MOD - 2);
12        for (int i = 2 * n - 1;i > 0;--i) inv[i] = inv[i + 1] * (i + 1);
13    }
14    Z <T,MOD> f (int x) {return fac[x];}
15    Z <T,MOD> inv_f (int x) {return inv[x];}
16    Z <T,MOD> comb (int x,int y)
17    {
18        if (x < 0 || y < 0 || y > x) return 0;
19        else return fac[x] * inv[x - y] * inv[y];
20    }
21    Z <T,MOD> arr (int x,int y)
22    {
23        if (x < 0 || y < 0 || y > x) return 0;
24        else return fac[x] * inv[x - y];
25    }
26    Z <T,MOD> catalan (int x) {return comb (2 * x,x) / (x + 1);}
27 };

```

3.5 The Sieve of Primes & Euler's Totient Function & Möbius Function

```

1 class Prime
2 {
3     int mx;
4     vector <int> p,phi,mu;

```

```

5 void pre ()
6 {
7     vector <int> fl (mx + 1,0);
8     phi[1] = mu[1] = 1;
9     for (int i = 2;i <= mx;++i)
10    {
11        if (!fl[i]) p.push_back (i),phi[i] = i - 1,mu[i] = -1;
12        for (auto v : p)
13        {
14            if (i * v > mx) break;
15            fl[i * v] = 1;
16            if (i % v == 0) {phi[i * v] = phi[i] * v,mu[i * v] = 0;break;}
17            phi[i * v] = phi[i] * (v - 1);mu[i * v] = -mu[i];
18        }
19    }
20 }
21 public :
22 Prime (int mx) : mx (mx),phi (mx + 1,0),mu (mx + 1,0) {pre ();}
23 auto get_prime () {return p;}
24 auto get_phi () {return phi;}
25 auto get_mu () {return mu;}
26 };

```

3.6 Exgcd

```

1 ll exgcd (ll a,ll b,ll &x,ll &y)
2 {
3     if (!b) {x = 1;y = 0;return abs (a);}
4     ll d = exgcd (b,(a % b + b) % b,y,x);
5     y -= a / b * x;
6     return d;
7 }

```

3.7 CRT

```

1 ll CRT ()
2 {
3     //ans % a[i] = b[i] if gcd (a[i],a[j]) = 1
4     ll sum = 1,ans = 0;
5     for (int i = 1;i <= n;++i) sum *= a[i];
6     for (int i = 1;i <= n;++i)
7     {
8         ll x,y,tmp = sum / a[i];
9         exgcd (tmp,a[i],x,y);
10        ans = (ans + tmp * x * b[i]) % sum;
11    }
12    return (ans + sum) % sum;
13 }

```

3.8 Lagrange Interpolation

```

1 ll Lagrange (ll num)
2 {
3     ll ans = 0;
4     for (int i = 1;i <= n;++i)
5     {
6         ll p = y[i] % MOD,q = 1;
7         for (int j = 1;j <= n;++j)

```

```

8      {
9          if (i == j) continue;
10         p = p * (num - x[j]) % MOD;
11         q = q * (x[i] - x[j]) % MOD;
12     }
13     ans = (ans + p * qpow (q,MOD - 2) % MOD + MOD) % MOD;
14 }
15 return ans;
16 }

```

3.9 Gaussian Elimination

```

1 vector <vector <double>> a (n + 1,vector <double> (n + 2)); // n * (n + 1)
2 for (int i = 1;i <= n;++i)
3 {
4     int p = cur,ok = 0;
5     while (p <= n)
6     {
7         if (a[p][i] != 0) {ok = 1;break;}
8         ++p;
9     }
10    if (!ok) continue;
11    for (int j = i;j <= n + 1;++j) swap (a[p][j],a[cur][j]);
12    for (int j = n + 1;j >= i;--j) a[cur][j] /= a[cur][i];
13    for (int j = 1;j <= n;++j)
14    {
15        if (j == cur) continue;
16        for (int k = n + 1;k >= i;--k) a[j][k] -= a[cur][k] * a[j][i];
17    }
18    ++cur;
19 }
20 if (cur <= n) puts ("No Solution"); //0 solution or infinte solutions
21 else {for (int i = 1;i <= n;++i) printf ("%6lf\n",a[i][n + 1]);}

```

3.10 FFT

```

1 using cd = complex <double>;
2 class FFT
3 {
4     const double Pi = acos (-1);
5     vector <int> rev;int n = 1;
6     void fft (vector <cd> &a,int op)
7     {
8         for (int i = 0;i < n;++i)
9             if (i < rev[i]) swap (a[i],a[rev[i]]);
10        for (int len = 1;len < n;len <<= 1)
11        {
12            cd wn (cos (Pi / len),op * sin (Pi / len));
13            for (int i = 0;i < n;i += len << 1)
14            {
15                cd w (1);
16                for(int j = 0;j < len;++j)
17                {
18                    cd u = a[i + j],v = a[i + j + len] * w;
19                    a[i + j] = u + v;a[i + j + len] = u - v;
20                    w *= wn;
21                }
22            }
23        }
24        if (op == -1) {for (auto &x : a) x /= n;}

```

```

25 }
26 public :
27 vector <double> conv (vector <cd> a,vector <cd> b)
28 {
29     int L = 0,tot = (int)a.size () + (int)b.size () - 1;n = 1;
30     while (n < tot) n <<= 1,++L;
31     rev.resize (n);a.resize (n);b.resize (n);
32     for (int i = 0;i < n;++i) rev[i] = (rev[i >> 1] >> 1) | ((i & 1) << (L - 1)
33 );
34     fft (a,1);fft (b,1);
35     for (int i = 0;i < n;++i) a[i] = a[i] * b[i];
36     fft (a,-1);
37     vector <double> ans (tot);
38     for (int i = 0;i < tot;++i) ans[i] = a[i].real ();
39     return ans;
40 };

```

3.11 NTT

```

1 template <int MOD = 998244353>
2 class NTT
3 {
4     vector <int> rev; int n = 1;
5     const int P = MOD;//998244353 469762049 500000000001507329LL 4179340454199820289
6     const int G = 3;
7     //1945555039024054273 for G = 5
8     int qpow(int x, int y)
9     {
10         int res = 1;
11         while (y)
12         {
13             if (y & 1) res = 1ll * res * x % P;
14             x = 1ll * x * x % P;
15             y >>= 1;
16         }
17         return res;
18     }
19     void init_rev(int tot)
20     {
21         int L = 0; n = 1;
22         while (n < tot) n <<= 1, ++L;
23         rev.resize(n);
24         for (int i = 0; i < n; ++i) rev[i] = (rev[i >> 1] >> 1) | ((i & 1) << (L -
25 1));
26     }
27     void ntt(vector <int> &a, int op)
28     {
29         for (int i = 0; i < n; ++i)
30             if (i < rev[i]) swap (a[i] ,a[rev[i]]);
31         for (int len = 1; len < n; len <<= 1)
32         {
33             int wn = qpow(G, (P - 1) / (len << 1));
34             if (op == -1) wn = qpow (wn, P - 2);
35             for (int i = 0; i < n; i += len << 1)
36             {
37                 int w = 1;
38                 for(int j = 0; j < len; ++j)
39                 {
40                     int u = a[i + j] % P, v = 1ll * a[i + j + len] * w % P;
41                     a[i + j] = (0ll + u + v) % P;
42                     a[i + j + len] = (0ll + u - v + P) % P;
43                 }
44                 w = 1ll * w * wn % P;
45             }
46             len <<= 1;
47         }
48     }
49 };

```

```

42         w = 1ll * w * wn % P;
43     }
44 }
45 }
46 if (op == -1)
47 {
48     int inv_n = qpow(n, P - 2);
49     for (auto &x : a) x = 1ll * x * inv_n % P;
50 }
51 }
52 public:
53 vector<int> Conv(vector<int> a, vector<int> b)
54 {
55     int tot = (int)a.size() + (int)b.size() - 1;
56     init_rev(tot);
57     a.resize(n); b.resize(n);
58     ntt(a, 1); ntt(b, 1);
59     for (int i = 0; i < n; ++i) a[i] = 1ll * a[i] * b[i] % P;
60     ntt(a, -1); a.resize(tot);
61     return a;
62 }
63 vector<int> Inv(vector<int> a, int m)
64 {
65     int inv = qpow(a[0], P - 2);
66     vector<int> b{inv};
67     for (int len = 2; (len >> 1) < m; len <= 1)
68     {
69         int sz = min((int)a.size(), len);
70         vector<int> f(a.begin(), a.begin() + sz);
71         init_rev(len <= 1);
72         f.resize(n);
73         vector<int> h = b; h.resize(n);
74         ntt(f, 1); ntt(h, 1);
75         for (int i = 0; i < n; ++i)
76             h[i] = 1ll * h[i] * (2ll - 1ll * f[i] * h[i] % P + P) % P;
77         ntt(h, -1);
78         b.assign(h.begin(), h.begin() + min(len, m));
79     }
80     b.resize(m);
81     return b;
82 }
83 pair<vector<int>, vector<int>> Divmod(vector<int> a, vector<int> b)
84 {
85     int n_sz = a.size(), m_sz = b.size();
86     if (n_sz < m_sz) return {{0}, a};
87     int dq = n_sz - m_sz + 1;
88     vector<int> ra = a, rb = b;
89     reverse(ra.begin(), ra.end()); reverse(rb.begin(), rb.end());
90     ra.resize(dq); rb.resize(dq);
91     vector<int> inv_rb = Inv(rb, dq);
92     vector<int> q = Conv(ra, inv_rb);
93     q.resize(dq);
94     reverse(q.begin(), q.end());
95     vector<int> qb = Conv(q, b);
96     vector<int> r(m_sz - 1);
97     for (int i = 0; i < m_sz - 1; ++i)
98         r[i] = (0ll + a[i] - (i < (int)qb.size() ? qb[i] : 0) + P) % P;
99     return {q, r};
100 }
101 vector<int> Sqrt(vector<int> a, int m) // b[0] = 1
102 {
103     vector<int> b{1};
104     int inv2 = qpow(2, P - 2);

```

```

105     for (int len = 2; (len >> 1) < m; len <= 1)
106     {
107         int sz = min((int)a.size(), len);
108         vector<int> f(a.begin(), a.begin() + sz);
109         vector<int> inv_b = Inv(b, len);
110         init_rev(len <= 1);
111         f.resize(n); inv_b.resize(n);
112         ntt(f, 1); ntt(inv_b, 1);
113         for (int i = 0; i < n; ++i)
114             f[i] = 1ll * f[i] * inv_b[i] % P;
115         ntt(f, -1);
116         b.resize(len);
117         for (int i = 0; i < len; ++i)
118             b[i] = 1ll * (b[i] + f[i]) % P * inv2 % P;
119     }
120     b.resize(m);
121     return b;
122 }
123 };

```

3.12 FWT

```

1  template<typename T>
2  class FWT
3  {
4  public:
5      const int P = 998244353;
6      const ll inv_2 = 499122177; // 5000000004 for 1e9 + 7
7  private:
8      void Or (vector<T> &F, int op = 1)
9      {
10         int n = F.size ();
11         for (int i = 2; i <= n; i <= 1)
12         {
13             int o = i >> 1;
14             for (int j = 0; j < n; j += i)
15                 for (int k = 0; k < o; ++k) F[j + k + o] = (F[j + k + o] + F[j + k] *
16 op % P) % P;
17         }
18     }
19     void And (vector<T> &F, int op = 1)
20     {
21         int n = F.size ();
22         for (int i = 2; i <= n; i <= 1)
23         {
24             int o = i >> 1;
25             for (int j = 0; j < n; j += i)
26                 for (int k = 0; k < o; ++k) F[j + k] = (F[j + k] + F[j + k + o] * op
27 % P) % P;
28         }
29     }
30     void Xor (vector<T> &F, int op = 1)
31     {
32         int n = F.size ();
33         for (int i = 2; i <= n; i <= 1)
34         {
35             int o = i >> 1;
36             for (int j = 0; j < n; j += i)
37             {
38                 for (int k = 0; k < o; ++k)
39                 {
40                     F[j + k] = (F[j + k] + F[j + k + o]) % P;
41                     F[j + k + o] = (F[j + k] - F[j + k + o]) % P;
42                 }
43             }
44         }
45     }
46 };

```

```

39         F[j + k + o] = (F[j + k] - 2 * F[j + k + o] % P + P) % P;
40         F[j + k] = F[j + k] * op % P;
41         F[j + k + o] = F[j + k + o] * op % P;
42     }
43 }
44 }
45 }
46 vector <T> mul (vector <T> A,vector <T> B)
47 {
48     int n = A.size ();
49     vector <T> C (n);
50     for (int i = 0;i < n;++i) C[i] = A[i] * B[i] % P;
51     return C;
52 }
53 public :
54 FWT () {}
55 vector <T> calc_Or (vector <T> A,vector <T> B)
56 {
57     Or (A);Or (B);
58     auto C = mul (A,B);
59     Or (C,P - 1);
60     return C;
61 }
62 vector <T> calc_And (vector <T> A,vector <T> B)
63 {
64     And (A);And (B);
65     auto C = mul (A,B);
66     And (C,P - 1);
67     return C;
68 }
69 vector <T> calc_Xor (vector <T> A,vector <T> B)
70 {
71     Xor (A);Xor (B);
72     auto C = mul (A,B);
73     Xor (C,inv_2);
74     return C;
75 }
76 };

```

4 Computational Geometry

```

1 using LD = long double;
2 const LD pi = acos (-1.0);
3 const LD eps = 1e-8L;
4 int dcmp (LD x) {return x < -eps ? -1 : (x > eps ? 1 : 0);}
5 struct Point {LD x,y;Point (LD x = 0,LD y = 0) : x (x),y (y) {};}
6 struct Circle {Point O;LD r;Circle (Point O = Point (),LD r = 0) : O (O),r (r) {};}
7 typedef Point Vector;
8 Vector operator + (Vector A,Vector B) {return Vector (A.x + B.x,A.y + B.y);}
9 Vector operator - (Vector A,Vector B) {return Vector (A.x - B.x,A.y - B.y);}
10 Vector operator * (Vector A,LD k) {return Vector (A.x * k,A.y * k);}
11 Vector operator / (Vector A,LD k) {return Vector (A.x / k,A.y / k);}
12 bool operator == (Vector A,Vector B) {return dcmp (A.x - B.x) == 0 && dcmp (A.y - B
    .y) == 0;}
13 bool operator != (Vector A,Vector B) {return !(A == B);}
14
15 LD dot (Vector A,Vector B) {return A.x * B.x + A.y * B.y;}
16 LD dis (Point A,Point B) {return sqrt ((A.x - B.x) * (A.x - B.x) + (A.y - B.y) * (A
    .y - B.y));}
17 LD cross (Vector A,Vector B) {return A.x * B.y - A.y * B.x;} // A -> B counter-
    clockwise if cross (A,B) > 0

```

```

18 LD len (Point A) {return sqrt (A.x * A.x + A.y * A.y);}
19 LD angle (Vector A,Vector B) {return acos (dot (A,B) / (len (A) * len (B)));}
20 Vector proj (Vector A,Vector B) {return A * (dot (A,B) / dot (A,A));} // project
    onto A
21 Point foot (Point P,Point A,Point B) {Vector AP = P - A,AB = B - A;return A + proj
    (AB,AP);} // foot
22 Point reflect (Point P,Point A,Point B) {Point F = foot (P,A,B);return F * 2 - P;}
    // symmetry point
23 Point rotate (Point P,LD theta) {return (Point){P.x * cos (theta) - P.y * sin (
    theta),P.x * sin (theta) + P.y * cos (theta)}}; // counterclockwise
24 bool chk_on_line (Point P,Point A,Point B) {return dcmp (cross (P - A,B - A)) ==
    0;} // judge whether on line AB
25 bool chk_on_seg (Point P,Point A,Point B) {return chk_on_line (P,A,B) && dcmp (dot
    (P - A,P - B)) <= 0;} // judge whether on segment AB
26 LD dis_seg (Point P,Point A,Point B) // the distance from point P to segment AB.
27 {
28     if (dcmp (dot (B - A,P - A)) < 0) return dis (P,A);
29     if (dcmp (dot (A - B,P - B)) < 0) return dis (P,B);
30     return fabs (cross (P - A,P - B)) / dis (A,B);
31 }
32 Point ll_inter_pt (Point A,Point B,Point C,Point D) {return A + (B - A) * cross (C
    - A,D - C) / cross (B - A,D - C);}
33 bool chk_ll_inter (Point A,Point B,Point C,Point D) {return dcmp (cross (B - A,D -
    C)) != 0;} // line - line
34 bool chk_ls_inter (Point A,Point B,Point C,Point D) {return chk_on_seg (ll_inter_pt
    (A,B,C,D),C,D);} // The intersection of AB(line) and CD (line) is on the CD (
    seg).
35 bool chk_ss_inter (Point A,Point B,Point C,Point D) // seg - seg
36 {
37     LD c1 = cross (B - A,C - A),c2 = cross (B - A,D - A);
38     LD d1 = cross (D - C,A - C),d2 = cross (D - C,B - C);
39     if (dcmp (c1) * dcmp (c2) < 0 && dcmp (d1) * dcmp (d2) < 0) return true;
40     if (dcmp(c1) == 0 && chk_on_seg (C,A,B)) return true;
41     if (dcmp(c2) == 0 && chk_on_seg (D,A,B)) return true;
42     if (dcmp(d1) == 0 && chk_on_seg (A,C,D)) return true;
43     if (dcmp(d2) == 0 && chk_on_seg (B,C,D)) return true;
44     return false;
45 }
46 bool SameSide (Point A,Point B,Point C,Point D) {return cross (A - C,D - C) * cross
    (D - C,B - C) < 0;} // Point A,B Line C,D
47
48 LD area (vector <Point> &P)
49 {
50     int n = P.size ();
51     LD res = 0;
52     for (int i = 0;i < n;++i) res += cross (P[i],P[(i + 1) % n]);
53     return res / 2.0;
54 }
55 bool is_convex (vector <Point> &P)
56 {
57     int n = P.size ();
58     for(int i = 0;i < n - 1;++i)
59         if (dcmp (cross (P[i + 1] - P[i],P[(i + 2) % n] - P[i])) < 0) return false;
60     return true;
61 }
62 int in_Poly (vector <Point> &P,Point A) // 0 (n) for any polygons
63 {
64     int cnt = 0,n = P.size ();
65     for (int i = 0;i < n;++i)
66     {
67         int j = (i + 1) % n;
68         if (chk_on_seg (A,P[i],P[j])) return 2;// on the edge

```



```

69     if (A.y >= min (P[i].y,P[j].y) && A.y < max (P[i].y,P[j].y)) // the
intersection is on the right
70         cnt += dcmp (((A.y - P[i].y) * (P[j].x - P[i].x) / (P[j].y - P[i].y) +
P[i].x) - A.x) > 0;
71     }
72     return cnt & 1;
73 }
74 int in_convex_Poly (vector <Point> &P,Point A) // 0 (log n) only for convex
polygons
75 {
76     int n = P.size ();
77     if (chk_on_seg (A,P[0],P[1]) || chk_on_seg (A,P[0],P[n - 1])) return 2;
78     if (dcmp (cross (P[n - 1] - P[0],A - P[0])) > 0 || dcmp (cross (P[1] - P[0],A -
P[0])) < 0) return 0;
79     int l = 1,r = n - 2,res = -1;
80     while (l <= r)
81     {
82         int mid = (l + r) >> 1;
83         if (dcmp (cross (P[mid] - P[0],A - P[0])) >= 0) res = mid,l = mid + 1;
84         else r = mid - 1;
85     }
86     if (chk_on_seg (A,P[res],P[res + 1])) return 2;
87     if (dcmp (cross (P[res + 1] - P[res],A - P[res])) < 0) return 0;
88     return 1;
89 }
90 auto convex_hull (vector <Point> &P) // strict convex hull (<= 0)
91 {
92     int n = P.size ();
93     sort (P.begin (),P.end (),[] (Point &x,Point &y) {return x.x == y.x ? x.y < y.y
: x.x < y.x;});
94     vector <Point> hull;
95     hull.resize (2 * n + 1);
96     int k = 0;
97     for (int i = 0;i < n;++i)
98     {
99         while (k >= 2 && dcmp (cross (hull[k - 1] - hull[k - 2],P[i] - hull[k - 2])
) <= 0) --k;
100         hull[k++] = P[i];
101     }
102     for (int i = n - 2,t = k;i >= 0;--i)
103     {
104         while (k > t && dcmp (cross (hull[k - 1] - hull[k - 2],P[i] - hull[k - 2]))
<= 0) --k;
105         hull[k++] = P[i];
106     }
107     hull.resize (k - 1);
108     return hull;
109 }
110 LD diameter (vector <Point> &P)
111 {
112     int n = P.size ();
113     if (n <= 1) return 0;
114     if (n == 2) return len (P[1] - P[0]);
115     LD res = 0;
116     for (int i = 0,j = 2;i < n;++i)
117     {
118         while (dcmp (cross (P[(i + 1) % n] - P[i],P[j] - P[i]) - cross (P[(i + 1) %
n] - P[i],P[(j + 1) % n] - P[i])) <= 0) j = (j + 1) % n;
119         res = max (res,max (len (P[j] - P[i]),len (P[j] - P[(i + 1) % n])));
120     }
121     return res;
122 }
123 template <typename Line = pair <Point, Point>>

```

```

124 LD half_plane (vector <Line> &Vec)
125 {
126     int n = Vec.size ();
127     auto get_angle = [&] (Line line) -> LD {return atan2 (line.second.y - line.
128     first.y,line.second.x - line.first.x);};
129     sort (Vec.begin (),Vec.end (),[&] (Line A,Line B)
130     {
131         LD angA = get_angle (A),angB = get_angle (B);
132         return fabs (angA - angB) > eps ? angA < angB : cross (A.second - A.first,B
133         .second - A.first) < 0;
134     });
135     int h = 1,t = 0;
136     auto check = [&] (Line x,Line y,Line z) -> bool
137     {
138         Point P = ll_inter_pt (y.first,y.second,z.first,z.second);
139         return dcmp (cross (x.second - x.first,P - x.first)) < 0;
140     };
141     vector <Line> q (n + 10);q[0] = Vec[0];
142     for (int i = 1;i < n;++i)
143     {
144         if (get_angle (Vec[i]) - get_angle (Vec[i - 1]) < eps) continue;
145         while (h < t && check (Vec[i],q[t],q[t - 1])) --t;
146         while (h < t && check (Vec[i],q[h],q[h + 1])) ++h;
147         q[0] = Vec[i];
148     }
149     while (h < t && check (q[h],q[t],q[t - 1])) --t;
150     q[0] = q[h];
151     vector <Point> p;
152     for (int i = h;i < t;++i) p.push_back (ll_inter_pt (q[i].first,q[i].second,q[i
153     + 1].first,q[i + 1].second));
154     return area (p);
155 }
156 vector <Point> minkowski (vector <Point> &vecA,vector <Point> &vecB)
157 {
158     int n = vecA.size (),m = vecB.size ();
159     vector <Point> A (n),B (m),C {vecA[0] + vecB[0]};
160     for (int i = 0;i < n;++i) A[i] = vecA[(i + 1) % n] - vecA[i];
161     for (int i = 0;i < m;++i) B[i] = vecB[(i + 1) % m] - vecB[i];
162     int posa = 0,posb = 0;
163     while (posa < n || posb < m)
164     {
165         if (posa == n) C.push_back (C.back () + B[posb++]);
166         else if (posb == m) C.push_back (C.back () + A[posa++]);
167         else if (dcmp (cross (A[posa],B[posb])) >= 0) C.push_back (C.back () + A[
168         posa++]);
169         else C.push_back (C.back () + B[posb++]);
170     }
171     return convex_hull (C);
172 }
173 bool chk_convex_inter_strict (vector <Point> A, vector <Point> B) // Check if two
174     convex polygons strictly intersect
175 {
176     vector <Point> negB;
177     for (auto p : B) negB.push_back (Point (-p.x, -p.y));
178     negB = convex_hull (negB);
179     vector <Point> C = minkowski (A, negB);
180     if (C.size () < 3) return false;
181     Point origin (0, 0);
182     return in_convex_Poly (C, origin) == 1;
183 }
184 bool chk_in_cir (Circle C,Point P) {return dcmp (len (P - C.O) - C.r) <= 0;}

```

```

181 Point get_cir_pt (Circle C,LD theta) {return {C.O.x + C.r * cos (theta),C.O.y + C.r
    * sin (theta)}};
182 int chk_lc_inter (Point A,Point B,Circle C)
183 {
184     LD d = dis_seg (C.O,A,B);
185     if (dcmp (d - C.r) == 0) return 0; // tangent
186     if (dcmp (d - C.r) > 0) return -1; // separation
187     return 1; // intersection
188 }
189 int chk_cc_inter (Circle A,Circle B) // the number of tagent lines
190 {
191     LD d = len (A.O - B.O);
192     if (dcmp (A.r + B.r - d) < 0) return 4; // externally separate
193     if (dcmp (A.r + B.r - d) == 0) return 3; // externally tangent
194     if (dcmp (fabs (A.r - B.r) - d) == 0) return 1; // internally tangent
195     if (dcmp (fabs (A.r - B.r) - d) > 0) return 0; // one circle inside the other
196     return 2; // intersection
197 }
198 pair <Point,Point> lc_inter (Point A,Point B,Circle C)
199 {
200     Point F = foot (C.O,A,B);LD d = dis (C.O,F);
201     Vector E = (B - A) / dis (A,B);
202     Point P1 = F - E * sqrt (C.r * C.r - d * d);
203     Point P2 = F + E * sqrt (C.r * C.r - d * d);
204     return {P1,P2};
205 }
206 pair <Point,Point> cc_inter (Circle A,Circle B)
207 {
208     Vector k = B.O - A.O;
209     LD d = len (k);
210     LD alpha = atan2 (k.y,k.x),beta = acos ((A.r * A.r + d * d - B.r * B.r) / (2 *
    A.r * d));
211     Point P1 = get_cir_pt (A,alpha - beta),P2 = get_cir_pt (A,alpha + beta);
212     return {P1,P2};
213 }
214 pair <Point,Point> tan_cir (Point P,Circle C)
215 {
216     LD d = len (C.O - P),theta = asin (C.r / d);
217     Vector E = (C.O - P) / d;
218     Vector P1 = P + (rotate (E,theta) * sqrt (d * d - C.r * C.r));
219     Vector P2 = P + (rotate (E,-theta) * sqrt (d * d - C.r * C.r));
220     return {P1,P2};
221 }
222 Circle tri_incir (Point A,Point B,Point C)
223 {
224     LD a = dis (B,C),b = dis (A,C),c = dis (A,B);
225     Point O = (A * a + B * b + C * c) / (a + b + c);
226     return {O,dis_seg (O,A,B)};
227 }
228 Circle tri_circum (Point A,Point B,Point C)
229 {
230     LD Bx = B.x - A.x,By = B.y - A.y,Cx = C.x - A.x,Cy = C.y - A.y;
231     LD D = 2 * (Bx * Cy - By * Cx);
232     LD x = (Cy * (Bx * Bx + By * By) - By * (Cx * Cx + Cy * Cy)) / D + A.x;
233     LD y = (Bx * (Cx * Cx + Cy * Cy) - Cx * (Bx * Bx + By * By)) / D + A.y;
234     Point P (x,y);
235     return Circle (P,dis (A,P));
236 }
237 auto get_tangents (Circle A,Circle B)
238 {
239     vector <pair <Point,Point>> tangents;
240     LD d = len (A.O - B.O),dif = A.r - B.r,sum = A.r + B.r;
241     if (dcmp (d - fabs (dif)) < 0) return tangents;

```

```

242 LD base = atan2 (B.O.y - A.O.y,B.O.x - A.O.x);
243 if (dcmp (d - fabs (dif)) == 0)
244 {
245     tangents.push_back ({get_cir_pt (A,base + (A.r < B.r ? pi : 0)),get_cir_pt
(A,base + (A.r < B.r ? pi : 0))});
246     return tangents;
247 }
248 LD theta = acos (dif / d);
249 tangents.push_back ({get_cir_pt (A,base + theta),get_cir_pt (B,base + theta)});
250 tangents.push_back ({get_cir_pt (A,base - theta),get_cir_pt (B,base - theta)});
251 if (dcmp (d - sum) == 0) tangents.push_back ({get_cir_pt (A,base),get_cir_pt (A
,base)});
252 if (dcmp (d - sum) > 0)
253 {
254     theta = acos (sum / d);
255     tangents.push_back ({get_cir_pt (A,base + theta),get_cir_pt (B,base + theta
+ pi)});
256     tangents.push_back ({get_cir_pt (A,base - theta),get_cir_pt (B,base - theta
+ pi)});
257 }
258 return tangents;
259 }
260 LD tri_cir_area (Point A,Point B,Circle C)
261 {
262     Vector OA = A - C.O,OB = B - C.O;
263     LD S = cross (OA,OB),sign = dcmp (cross (OA,OB)) > 0 ? 1 : -1;
264     bool da = dcmp (len (OA) - C.r) < 0,db = dcmp (len (OB) - C.r) < 0;
265     if (dcmp (S) == 0) return 0;
266     if (da && db) return S * 0.5; // triangle
267     if (!da && !db)
268     {
269         if (chk_lc_inter (A,B,C) == 1)// arc + triangle + arc
270         {
271             auto [P1,P2] = lc_inter (A,B,C);
272             Vector OP1 = P1 - C.O,OP2 = P2 - C.O;
273             if (dis (A,P1) > dis (A,P2)) swap (P1,P2);
274             return cross (OP1,OP2) * 0.5 + sign * 0.5 * C.r * C.r * (angle (OA,OP1)
+ angle (OB,OP2));
275         }
276         else return sign * 0.5 * C.r * C.r * angle (OA,OB); // arc
277     }
278     else // triangle + arc
279     {
280         auto [P1,P2] = lc_inter (A,B,C);
281         if (chk_on_seg (P2,A,B)) swap (P1,P2);
282         Vector OP1 = P1 - C.O;
283         if (dcmp (len (OA) - C.r) < 0) return cross (OA,OP1) * 0.5 + sign * 0.5 * C
.r * C.r * angle (OP1,OB);
284         else return cross (OP1,OB) * 0.5 + sign * 0.5 * C.r * C.r * angle (OP1,OA);
285     }
286 }
287 LD cc_area (Circle C1,Circle C2)
288 {
289     int op = chk_cc_inter (C1,C2);
290     if (op <= 1) return pi * min (C1.r,C2.r) * min (C1.r,C2.r);
291     else if (op == 4) return 0;
292     else
293     {
294         LD d = dis (C1.O,C2.O);
295         LD alpha = 2 * acos ((C1.r * C1.r - C2.r * C2.r + d * d) / (2 * C1.r * d));
296         LD beta = 2 * acos ((C2.r * C2.r - C1.r * C1.r + d * d) / (2 * C2.r * d));
297         return 0.5 * (C1.r * C1.r * (alpha - sin (alpha)) + C2.r * C2.r * (beta -
sin (beta)));

```

298
299

```
}  
}
```

5 String

5.1 Hash

```
1 template <unsigned long long base = 13331>
2 class Hash
3 {
4     using u64 = unsigned long long;
5     vector <u64> pw,hsh;
6     public:
7     Hash (char *s)
8     {
9         int n = strlen (s);
10        pw.assign (n + 1,0),hsh.assign (n + 1,0);
11        pw[0] = 1;hsh[0] = 0;
12        for (int i = 1;i <= n;++i) pw[i] = pw[i - 1] * base,hsh[i] = hsh[i - 1] *
13        base + s[i - 1];
14    }
15    u64 get (int l,int r) {return hsh[r] - hsh[l - 1] * pw[r - l + 1];}
16    u64 link (int l1,int r1,int l2,int r2) {return get (l1,r1) * pw[r2 - l2 + 1] +
17    get (l2,r2);}
18    bool same (int l1,int r1,int l2,int r2) {return get (l1,r1) == get (l2,r2);}
19};
```

5.2 KMP

```
1 class KMP
2 {
3     vector <int> fail,ans;
4     void getfail (char *s) // Match a with b
5     {
6         int len = strlen (s);
7         fail.assign (len + 1,0);
8         fail[0] = -1;
9         for (int i = 1;i < len;++i)
10        {
11            int cnt = fail[i - 1];
12            while (cnt >= 0 && s[cnt + 1] != s[i]) cnt = fail[cnt];
13            if (s[cnt + 1] == s[i]) ++cnt;
14            fail[i] = cnt;
15        }
16    }
17    public:
18    auto get_pos (char *s,char *t)
19    {
20        getfail (t);
21        int lens = strlen (s),lent = strlen (t),cnt = -1;
22        for (int i = 0;i < lens;++i)
23        {
24            while (cnt >= 0 && t[cnt + 1] != s[i]) cnt = fail[cnt];
25            if (t[cnt + 1] == s[i])
26            {
27                ++cnt;
28                if (cnt + 1 == lent) ans.push_back (i - lent + 2),cnt = fail[cnt];
29            }
30        }
31        return ans;
32    }
33};
```

5.3 Manacher

```
1 int Manacher (char *s)
2 {
3     int n = strlen (s), cnt = 0, r = 0, mid = 0, ans = 0;
4     vector <char> a (2 * n + 5); vector <int> p (2 * n + 5);
5     a[++cnt] = '!'; a[++cnt] = '#';
6     for (int i = 0; i < n; ++i) a[++cnt] = s[i], a[++cnt] = '#';
7     a[++cnt] = '~';
8     for (int i = 2; i < cnt; ++i)
9     {
10         if (i <= r) p[i] = min (r - i + 1, p[mid * 2 - i]);
11         else p[i] = 1;
12         while (a[i - p[i]] == a[i + p[i]]) ++p[i];
13         if (i + p[i] > r) r = i + p[i] - 1, mid = i;
14         ans = max (ans, p[i]);
15     }
16     return ans - 1;
17 }
```

5.4 Trie

5.4.1 Trie

```
1 struct Trie
2 {
3     int n, m, cnt; // m total len
4     vector <vector <int>> ch; vector <int> vis;
5     public :
6     Trie (int n, int m) : n (n), m (m), ch (m, vector <int> (26, 0)), vis (m) {cnt = 0;}
7     void insert (char *s)
8     {
9         int u = 0, len = strlen (s);
10        for (int i = 0; i < len; ++i) m
11        {
12            int c = s[i] - 'a';
13            if (!ch[u][c]) ch[u][c] = ++cnt;
14            u = ch[u][c];
15        }
16        ++vis[u];
17    }
18    int query (char *s)
19    {
20        int u = 0, len = strlen (s);
21        for (int i = 0; i < len; ++i)
22        {
23            int c = s[i] - 'a';
24            if (!ch[u][c]) return 0;
25            u = ch[u][c];
26        }
27        return vis[u];
28    }
29 };
```

5.4.2 01-Trie

```
1 class Trie
2 {
3     int n, cnt;
4     vector <vector <int>> ch;
```

```

5  vector <int> val,w;
6  public :
7  Trie (int n,int lg) : n (n),val (2 * lg * n + 1,0),w (2 * lg * n + 1,0),ch (2 *
   lg * n + 1,vector <int> (2,0)) {cnt = 1;}
8  void pushup (int u)
9  {
10     w[u] = val[u] = 0;
11     //w[u] Number of values (weights) on the edge between node u and its parent
   node
12     //val[u] XOR sum maintained by the subtree rooted at u
13     if (ch[u][0]) w[u] ^= w[ch[u][0]],val[u] ^= val[ch[u][0]] << 1;
14     if (ch[u][1]) w[u] ^= w[ch[u][1]],val[u] ^= (val[ch[u][1]] << 1) | w[ch[u]
   ][1];
15 }
16 void modify (int &u,int v,int dep)
17 {
18     if (!u) u = ++cnt;
19     w[u] ^= 1;
20     if (dep < 0) return ;
21     modify (ch[u][v & 1],v >> 1,dep - 1);
22     pushup (u);
23 }
24 void erase (int u,int v,int dep)
25 {
26     if (!u) return ;
27     w[u] ^= 1;
28     if (dep < 0) return ;
29     erase (ch[u][v & 1],v >> 1,dep - 1);
30     pushup (u);
31 }
32 void add (int u) // add 1 in [1,n]
33 {
34     swap (ch[u][0],ch[u][1]);
35     if (ch[u][0]) add (ch[u][0]);
36     pushup (u);
37 }
38 };

```

5.5 Aho-Corasick Automaton

```

1  class AC
2  {
3      int n,m,cnt;//m total len
4      vector <vector <int>> tr;vector <int> val;
5      public :
6      Trie (int n,int m) : n (n),m (m),tr (m + 1,vector <int> (26,0)),val (m + 1),
   fail (m + 1) {cnt = 1;}
7      void insert (char *s)
8      {
9          int n = strlen (s + 1),u = 1;
10         for (int i = 1;i <= N;++i)
11         {
12             if (!tr[u][s[i] - 'a']) tr[u][s[i] - 'a'] = ++cnt;
13             u = tr[u][s[i] - 'a'];
14         }
15         ++val[u];
16     }
17     void build ()
18     {
19         for (int i = 0;i < 26;++i) tr[0][i] = 1;
20         queue <int> q;q.push (1);
21         while (!q.empty ())

```



```

22     {
23         int u = q.front ();q.pop ();
24         for (int i = 0;i < 26;++i)
25         {
26             if (!tr[u][i]) tr[u][i] = tr[fail[u]][i];
27             else fail[tr[u][i]] = tr[fail[u]][i],q.push (tr[u][i]);
28         }
29     }
30 }
31 int query (char *s)
32 {
33     int n = strlen (s + 1),u = 1,res = 0;
34     for (int i = 1;i <= n;++i)
35     {
36         u = tr[u][s[i] - 'a'];
37         for (int j = u;j && ~val[j];j = fail[j]) res += val[j],val[j] = -1;
38     }
39     return res;
40 }
41 };
42 /*
43 s :
44 a
45 aa
46 aa
47 aaa
48 t :
49 aaaa
50 ans : 4
51 */

```

5.6 SA

```

1 struct SA
2 {
3     string s;
4     vector <int> sa,rk,ht;
5     // all 0-base including sa
6     // suffix order
7     // rank of suffix i
8     // LCP(sf[i], sf[i+1])
9     vector <int> tmp;
10    int n;
11    SA (string &t) {init (t);}
12    inline void tSort (int m) //size of char set (base sort)
13    {
14        vector<int> tmp1 (m + 1,0);
15        for (int i = 0;i < n;++i) ++tmp1[rk[tmp[i]]];
16        for (int i = 1;i <= m;++i) tmp1[i] += tmp1[i - 1];
17        for (int i = n - 1;i >= 0;--i) sa[--tmp1[rk[tmp[i]]]] = tmp[i];
18    }
19    inline void init (string &t)
20    {
21        s = t;n = s.size ();
22        sa.assign (n,0);rk.assign (n,0);ht.assign (n,0);tmp.assign (n,0);
23        for (int i = 0;i < n;++i) rk[i] = s[i],tmp[i] = i;
24        int m = 127;tSort (m);
25        for (int k = 1,p = 0;p < n;k <<= 1)
26        {
27            p = 0;
28            for (int i = n - k;i < n;++i) tmp[p++] = i;
29            for (int i = 0;i < n;++i)

```

```

30         if (sa[i] >= k) tmp[p++] = sa[i] - k;
31     tSort (m);
32     vector <int> old_rk = rk;
33     rk[sa[0]] = p = 1;
34     for (int i = 1; i < n; ++i)
35     {
36         int a = sa[i], b = sa[i - 1];
37         if (old_rk[a] == old_rk[b] && (a + k < n ? old_rk[a + k] : 0) == (b
+ k < n ? old_rk[b + k] : 0))
38             rk[a] = p;
39         else
40             rk[a] = ++p;
41     }
42     m = p;
43     if (p == n) break;
44 }
45 for (int i = 0; i < n; ++i) --rk[i];
46 int h = 0;
47 for (int i = 0; i < n; ++i)
48 {
49     if (rk[i] == 0) continue;
50     int j = sa[rk[i] - 1];
51     while (i + h < n && j + h < n && s[i + h] == s[j + h]) ++h;
52     ht[rk[i] - 1] = h;
53     if (h > 0) --h;
54 }
55 }
56 inline void clear ()
57 {
58     s.clear (); sa.clear (); rk.clear (); ht.clear (); tmp.clear ();
59     n = 0;
60 }
61 inline int size () const {return n;}
62 inline int operator[] (int p) const {return sa[p];}
63 };

```

5.7 SAM

```

1 class SAM
2 {
3     class node
4     {
5         public: int ch[26], len, fa;
6         node (const int &L = 0) {memset (ch, 0, sizeof (ch)); fa = 0; len = L;}
7     };
8     public:
9     vector <node> t; int lst;
10    void GetParentTree (vector <vector <int>> &G)
11    {
12        G.resize (t.size ());
13        for (unsigned i = 1; i < t.size (); ++i) G[t[i].fa].push_back (i);
14    }
15    void extend (const int &c)
16    {
17        int p = lst, np = lst = t.size ();
18        t.push_back (node (t[p].len + 1));
19        for (; p && !t[p].ch[c]; p = t[p].fa) t[p].ch[c] = np;
20        if (!p) t[np].fa = 1;
21        else
22        {
23            int v = t[p].ch[c];
24            if (t[v].len == t[p].len + 1) t[np].fa = v;

```

```

25         else
26         {
27             int nv = t.size ();t.push_back (t[v]);
28             t[nv].len = t[p].len + 1;
29             for (;p && t[p].ch[c] == v;p = t[p].fa) t[p].ch[c] = nv;
30             t[np].fa = t[v].fa = nv;
31         }
32     }
33 }
34 SAM () {t.assign (2,node ());lst = 1;}
35 inline void clear () {t.assign (2,node ());lst = 1;}
36 inline int next (int p,int c) {return t[p].ch[c];}
37 inline int Len (int p) {return t[p].len;}
38 };

```

6 Other

6.1 Fast I/O

```
1 #include <bits/stdc++.h>
2 using namespace std;
3 constexpr int MAX = 1e5 + 5;
4 constexpr int MOD = 1e9 + 7;
5 constexpr int INF = 0x3f3f3f3f;
6 using i64 = long long;
7 using u64 = unsigned long long;
8 using pii = pair <int, int>;
9 using vi = vector <int>;
10 using vii = vector <pii>;
11 inline int read()
12 {
13     int s = 0; int f = 1;
14     char ch = getchar();
15     while ((ch < '0' || ch > '9') && ch != EOF)
16     {
17         if (ch == '-') f = -1;
18         ch = getchar();
19     }
20     while (ch >= '0' && ch <= '9')
21     {
22         s = s * 10 + ch - '0';
23         ch = getchar();
24     }
25     return s * f;
26 }
27 inline void write(int x)
28 {
29     if (x < 0) {putchar ('-'); write(-x); return;}
30     if (x > 9) write(x / 10);
31     putchar(x % 10 + '0');
32 }
33 int main()
34 {
35     //freopen(".in", "r", stdin);
36     //freopen(".out", "w", stdout);
37     return 0;
38 }
```

6.2 Random Number Generator

```
1 int rd(int a, int b)
2 {
3     static std::mt19937 gen(std::random_device{}());
4     std::uniform_int_distribution <int> dist(a, b);
5     return dist(gen);
6 }
```

6.3 Execution

```
1 #!/bin/bash
2 ulimit -s 102400
3 g++ -std=c++17 -O2 -Wall "$1" -o main
4 ./main < in.txt > out.txt
```

```

1 # Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass
2 # new file : 'type nul > filename.cpp'
3 # run : '.\run.ps1 filename.cpp'
4 g++ -std=c++17 -O2 -Wall $args[0] -o main
5 cat in.txt | .\main | Out-File -FilePath out.txt

```

6.4 Comparision

```

1 #!/bin/bash
2 for ((i=1; ; i++)); do
3     ./gen > input.txt           # generator
4     ./code < input.txt > output1.txt # your code
5     ./std < input.txt > output2.txt  # std code
6     if diff output1.txt output2.txt; then # compare
7         echo "Test $i: Accepted"
8     else
9         echo "Test $i: Wrong Answer"
10        echo "Input:"
11        cat input.txt
12        echo "Your Output:"
13        cat output1.txt
14        echo "Standard Output:"
15        cat output2.txt
16        break
17    fi
18 done

```

6.5 Python

```

1 import sys
2 sys.set_int_max_str_digits (value) # set big number
3 sys.setrecursionlimit (deep) # set deeper recursion
4 lst = sorted (lst, key = lambda x : x[1]) # sort by the second element

```